



I302 - Aprendizaje Automático y Aprendizaje Profundo

Trabajo Práctico 4

Abril Diaco

9 de junio de 2026

Ingeniería en Inteligencia Artificial

Resumen

En este trabajo se aplicaron técnicas de reducción de dimensionalidad y clustering sobre un subconjunto del dataset Fashion-MNIST. Inicialmente se realizó un análisis exploratorio de los datos y un preprocesamiento basado en estandarización y partición estratificada en conjuntos de entrenamiento y evaluación. Posteriormente, se obtuvieron representaciones latentes mediante Principal Component Analysis (PCA) y un Autoencoder determinístico implementado en PyTorch, comparando ambos enfoques a través de métricas de reconstrucción.

Sobre las representaciones reducidas se aplicaron los algoritmos de clustering K-Means y Gaussian Mixture Models (GMM). La selección del número de clusters se realizó utilizando ganancias marginales, inercia, log-verosimilitud y *Silhouette score*, resultando en la elección de $K = 8$ para ambos métodos. Finalmente, se analizaron la composición, pureza y distribución de los clusters obtenidos, complementando el estudio mediante visualizaciones con t-SNE.

Los resultados muestran que PCA y el Autoencoder logran niveles de reconstrucción similares, con una ligera ventaja de PCA en términos de error cuadrático. Asimismo, los algoritmos de clustering consiguieron identificar parcialmente la estructura visual presente en los datos, separando con mayor facilidad categorías visualmente distintivas como bolsos, pantalones y calzado. Sin embargo, las clases correspondientes a distintas prendas superiores presentaron una mayor superposición, evidenciando las limitaciones inherentes del aprendizaje no supervisado para recuperar exactamente las categorías originales del dataset.

1. Introducción

El aprendizaje no supervisado abarca un conjunto de métodos cuyo propósito es detectar patrones, estructuras y relaciones en los datos sin contar con etiquetas durante la fase de entrenamiento. A diferencia del aprendizaje supervisado, que se centra en aprender una relación entre entradas y salidas conocidas, en este enfoque el modelo debe hallar regularidades en los datos a partir de sus propias características. Entre las tareas más comunes de este problema se encuentran el clustering y la reducción de dimensionalidad.

En este trabajo se estudió la aplicación conjunta de técnicas de reducción de dimensionalidad y algoritmos de clustering sobre el conjunto de datos *Fashion-MNIST*. Este dataset está compuesto por imágenes en escala de grises de 28×28 píxeles, correspondientes a distintas categorías de prendas de vestir. Cada imagen se representa como un vector de 784 características, donde cada componente indica la intensidad de un píxel. Aunque cada muestra posee una etiqueta asociada a su clase real, dichas etiquetas no fueron utilizadas durante el entrenamiento de los modelos, sino únicamente para facilitar la interpretación y evaluación de los resultados obtenidos.

Debido a la alta dimensionalidad de las imágenes, el costo computacional asociado a los algoritmos de agrupamiento puede resultar elevado. Por este motivo, se emplearon métodos de reducción de dimensionalidad para obtener representaciones más compactas de los datos. En particular, se utilizaron Principal Component Analysis (PCA) y Autoencoders Determinísticos para proyectar las imágenes a espacios latentes de menor dimensión, preservando en la medida de lo posible la información y estructura relevantes del conjunto de datos.

Posteriormente, las representaciones reducidas fueron utilizadas como entrada de los algoritmos de clustering K-Means y Gaussian Mixture Models (GMM), cuyo objetivo es agrupar observaciones similares sin conocimiento previo de sus clases reales. La salida de estos algoritmos consiste en una asignación de cada muestra a un cluster, ya sea de forma determinística en el caso de K-Means o probabilística en el caso de GMM.

Finalmente, se analizaron las agrupaciones obtenidas y se evaluó en qué medida los clusters descubiertos reflejan la estructura del dataset y su relación con las categorías originales de Fashion-MNIST.

2. Métodos

2.1 Conjunto de Datos y Preprocesamiento

El conjunto de datos utilizado corresponde a un subconjunto de 25000 muestras del dataset *Fashion-MNIST*, una base de datos compuesta por imágenes de artículos de indumentaria. Cada imagen se encuentra representada como un vector de 784 píxeles (28×28), con valores de intensidad dentro del intervalo $[0, 1]$. Además, cada muestra posee una etiqueta numérica asociada a su clase real: 0 corresponde a remera/top, 1 a pantalón, 2 a pulóver, 3 a vestido, 4 a abrigo, 5 a sandalia, 6 a camisa, 7 a zapatilla, 8 a bolso y 9 a bota. Dado que el objetivo del trabajo es aplicar técnicas de aprendizaje no supervisado, estas etiquetas fueron utilizadas únicamente para la interpretación y evaluación posterior de los resultados.

El dataset fue dividido en un conjunto de entrenamiento y otro de evaluación mediante un muestreo estratificado, garantizando que la proporción de muestras de cada clase sea similar en ambos conjuntos. En particular, el 80 % de las observaciones se destinó al entrenamiento y el 20 % restante a la evaluación.

Como parte del análisis exploratorio inicial, se visualizó una muestra aleatoria del dataset, la distribución de clases y la imagen promedio de algunas muestras. Además, se calcularon algunas estadísticas como el desvío y promedio de intensidad total entre imágenes. Este análisis permitió caracterizar visualmente el dataset antes de aplicar las técnicas de reducción de dimensionalidad y clustering.

Como se mencionó anteriormente, el análisis se realizó sobre representaciones de dimensión reducida. Por este motivo, antes de aplicar los métodos de reducción de dimensionalidad, las variables fueron estandarizadas. Para cada característica se aplicó la transformación

$$x_{\text{std}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

Posteriormente, la misma transformación fue aplicada al conjunto de evaluación utilizando los parámetros obtenidos durante el entrenamiento.

2.2 Técnicas de reducción de dimensionalidad

La reducción de dimensionalidad busca representar los datos en un espacio de menor dimensión preservando la mayor cantidad posible de información relevante. Estas técnicas permiten disminuir el costo computacional de los algoritmos posteriores, reducir el ruido y facilitar la visualización e interpretación de conjuntos de datos de alta dimensión.

En términos generales, el objetivo consiste en transformar cada observación original x en una representación latente z de menor dimensión, a partir de la cual pueda reconstruirse una aproximación $\hat{x}$ de los datos originales.

En este trabajo se implementaron dos enfoques para obtener representaciones latentes: Principal Component Analysis (PCA), un método lineal basado en proyecciones sobre direcciones de máxima varianza, y Autoencoders Determinísticos, modelos neuronales entrenados para minimizar el error de reconstrucción.

PCA

La idea de PCA es proyectar los datos sobre un subespacio a través de una transformación lineal, tal que las coordenadas sobre dicho subespacio conserven la mayor cantidad posible de variabilidad. Para ello, busca direcciones ortogonales denominadas *componentes principales*, ordenadas según la varianza que explican en los datos.

Dado un conjunto de datos centrado X , las componentes principales se obtuvieron mediante descomposición en valores singulares (SVD):

$$X = USV^T,$$

las cuales corresponden a las columnas de V . Una vez obtenidas, los datos se proyectaron sobre las primeras m componentes mediante

$$Z = XV_m,$$

donde V_m contiene únicamente las componentes principales seleccionadas.

La cantidad m de componentes se determinó a partir de la varianza acumulada explicada, seleccionando el menor valor de m capaz de preservar al menos el 90 % de la variabilidad presente en los datos originales. De esta manera, cada imagen queda representada mediante un vector de dimensión reducida z .

Finalmente, la reconstrucción aproximada de las observaciones originales se obtuvo proyectando nuevamente desde el espacio latente hacia el espacio de entrada:

$$\hat{X} = ZV_m^T.$$

Las representaciones reducidas obtenidas mediante PCA fueron posteriormente utilizadas como entrada para los algoritmos de clustering analizados en este trabajo.

Autoencoder determinístico

A diferencia de PCA, que busca una proyección lineal sobre direcciones de máxima varianza, los autoencoders determinísticos permiten aprender representaciones latentes mediante redes neuronales. Su objetivo es encontrar una función de codificación y una función de decodificación que permitan reconstruir las observaciones originales con la menor pérdida de información posible.

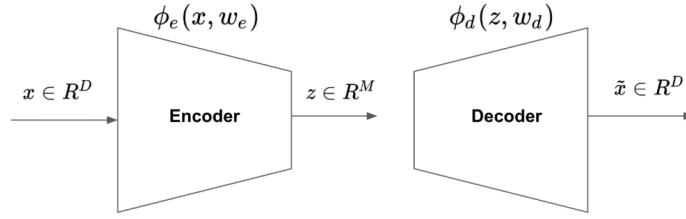


Figura 1: Arquitectura conceptual de un autoencoder. La información se comprime en un espacio latente y posteriormente se reconstruye para minimizar el error entre la entrada y la salida.

La arquitectura de un autoencoder, visualizada en la Figura 1, está compuesta por un encoder, que transforma una observación x en una representación latente z , y un decoder, que reconstruye una aproximación $\hat{x}$ de la entrada original. Matemáticamente,

$$z = \phi_e(x), \quad \hat{x} = \phi_d(z).$$

Los parámetros de ambas redes se ajustan minimizando el error de reconstrucción medido mediante MSE:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$

Con el objetivo de realizar una comparación justa con PCA, la dimensión del espacio latente se fijó igual a la obtenida mediante dicho método, es decir, de m dimensiones. De esta manera, ambos algoritmos operan bajo el mismo nivel de compresión.

El autoencoder fue implementado utilizando la librería `PyTorch`. La arquitectura elegida para el encoder consistió en dos capas ocultas, de 512 y 256 neuronas respectivamente, seguidas de una capa latente de dimensión m . El decoder utilizó la estructura inversa, reconstruyendo progresivamente la información hasta recuperar las 784 variables originales correspondientes a cada imagen.

El entrenamiento se realizó utilizando el optimizador Adam, un *batch-size* de 128 muestras, un *learning rate* de 0.001, funciones de activación ReLU y 30 épocas de entrenamiento.

Una vez finalizado el entrenamiento, las representaciones latentes generadas por el encoder fueron utilizadas como entrada para los algoritmos de clustering implementados en este trabajo.

2.3 K-Means

Para la etapa de clustering se trabajó con una submuestra estratificada de 3000 observaciones extraídas del conjunto de entrenamiento. Esta decisión se tomó con el objetivo de reducir el costo computacional de los algoritmos sin alterar significativamente la distribución original de las clases presentes en el dataset.

K-Means es un algoritmo de clustering cuyo objetivo es dividir un conjunto de datos en K grupos, asignando cada observación al cluster cuyo centroide se encuentre más cerca. La idea, entonces, es que los elementos pertenecientes a un mismo cluster sean similares entre sí, mientras que observaciones de distintos clusters estén alejadas.

Formalmente, el algoritmo busca minimizar la inercia

$$J = \sum_{i=1}^n \sum_{j=1}^K r_{ij} \|x_i - \mu_j\|^2,$$

donde $r_{ij} = 1$ si la observación i pertenece al cluster j , y 0 en caso contrario, mientras que μ_j representa el centroide del cluster j .

El algoritmo alterna entre la asignación de observaciones al centroide más cercano y la actualización de los centroides como el promedio de los puntos asignados, repitiendo el proceso hasta converger.

La función de costo se evaluó para valores de K comprendidos entre 5 y 15. La inicialización se realizó mediante K-Means++, que selecciona centroides iniciales alejados entre sí para mejorar la estabilidad del algoritmo.

2.4 Gaussian Mixture Model

Además de K-Means, se implementó un Gaussian Mixture Model (GMM). A diferencia de K-Means, que realiza asignaciones determinísticas, GMM modela los datos como una mezcla de distribuciones gaussianas y permite asignaciones probabilísticas.

La distribución de una observación se expresa como

$$p(x_i) = \sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j),$$

donde π_j , μ_j y Σ_j representan respectivamente el peso, la media y la covarianza de la componente j . En este trabajo se utilizaron matrices de covarianza diagonales.

Los parámetros se estimaron mediante el algoritmo Expectation-Maximization (EM). En el paso E se calcularon las responsabilidades

$$\gamma_{ij} = P(z_i = j | x_i) = \frac{\pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}{\sum_{\ell=1}^K \pi_\ell \mathcal{N}(x_i | \mu_\ell, \Sigma_\ell)}.$$

mientras que en el paso M se actualizaron los parámetros según

$$N_j = \sum_i \gamma_{ij}, \quad \pi_j^* = \frac{N_j}{N}, \quad \mu_j^* = \frac{\sum_i \gamma_{ij} x_i}{N_j}, \quad \Sigma_j^* = \frac{\sum_i \gamma_{ij} (x_i - \mu_j)(x_i - \mu_j)^T}{N_j}.$$

El procedimiento se repitió hasta la convergencia de la log-verosimilitud. Para mejorar la estabilidad numérica, los cálculos se realizaron en escala logarítmica y se incorporó regularización en las varianzas.

Antes de iniciar el algoritmo EM, fue necesario definir valores iniciales para los parámetros del modelo. Las medias de las componentes gaussianas se inicializaron seleccionando aleatoriamente K observaciones del conjunto de datos, mientras que los pesos se fijaron inicialmente iguales para todas las componentes, es decir, $\pi_j = \frac{1}{K}$. Por su parte, las matrices de covarianza diagonales se inicializaron utilizando la varianza global de cada característica en el conjunto de datos.

Al igual que en K-Means, el modelo se evaluó para valores de K entre 5 y 15.

2.5 Selección de K y Métricas de Evaluación

La calidad de reconstrucción de PCA y del Autoencoder se evaluó mediante el Error Cuadrático Medio (MSE), la Raíz del Error Cuadrático Medio (RMSE) y el Error Absoluto Medio (MAE):

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2, \quad RMSE = \sqrt{MSE}, \quad MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Para seleccionar el número de clusters se analizaron las ganancias marginales y el *Silhouette score*. En K-Means, las ganancias marginales se calcularon a partir de la reducción de la inercia al incrementar K , mientras que en GMM se utilizaron las mejoras sucesivas de la log-verosimilitud.

El *Silhouette score* se define como:

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)},$$

donde a_i es la distancia promedio al propio cluster y b_i la distancia promedio al cluster vecino más cercano. Valores próximos a 1 indican grupos compactos y bien separados.

Finalmente, se utilizó t-SNE para visualizar las representaciones latentes en dos dimensiones preservando relaciones locales de vecindad entre observaciones.

3. Resultados

3.1 Análisis Exploratorio

La distribución de clases del subconjunto utilizado se presenta en la Figura 2. Puede observarse que el conjunto se encuentra balanceado, con 2500 muestras por categoría. Esta característica resulta favorable para la evaluación de los algoritmos, ya que evita que determinadas clases dominen los resultados debido a una representación significativamente mayor.

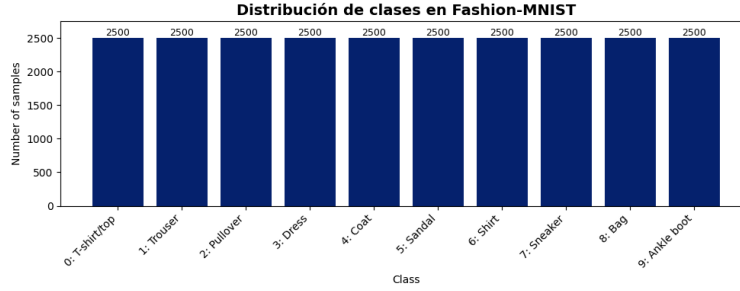


Figura 2: Cantidad de observaciones por clase en el subconjunto analizado de Fashion-MNIST. La distribución balanceada evita sesgos asociados a clases sobrerrepresentadas.

La Figura 3a muestra 15 imágenes aleatorias del dataset. Se observa que las categorías correspondientes al calzado presentan una mayor proporción de píxeles oscuros debido al espacio vacío que rodea al objeto. Asimismo, este mismo tipo de muestras, presenta una variabilidad considerable, donde aparecen diferencias importantes de formas y estilos.

Las imágenes promedio de cada clase se presentan en la Figura 3b. Estas permiten visualizar las características generales que distinguen a cada categoría. Algunas clases, como pantalón, zapatilla, bolso y bota, presentan siluetas claramente diferenciables. En contraste, categorías como remera/top, pulóver, abrigo y camisa tienen estructuras visuales similares.

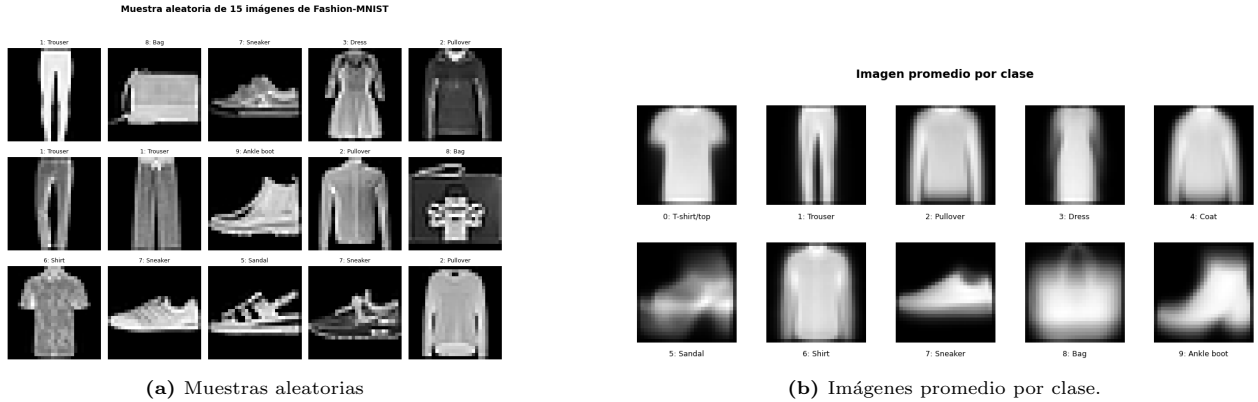


Figura 3: (a) Muestras aleatorias representativas de distintas categorías. (b) Imágenes promedio por clase, obtenidas promediando los píxeles de todas las observaciones de cada categoría.

Estas observaciones sugieren que ciertas categorías deberían resultar relativamente sencillas de separar mediante técnicas de clustering, mientras que otras podrían presentar mayores dificultades debido a sus similitudes visuales. En consecuencia, no necesariamente debe esperarse una correspondencia perfecta entre los clusters obtenidos y las etiquetas originales del dataset.

3.2 Reducción de Dimensionalidad

Aplicando PCA sobre el conjunto de entrenamiento, se determinó que eran necesarias 135 componentes principales para explicar el 90% de la varianza acumulada de los datos (ver Figura 4). Esta dimensión latente fue utilizada posteriormente tanto para PCA como para el Autoencoder, permitiendo comparar ambos métodos bajo el mismo nivel de compresión.

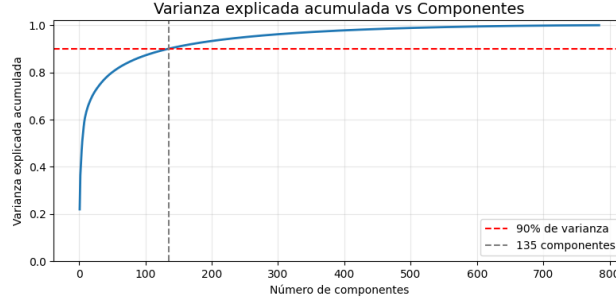


Figura 4: Varianza acumulada explicada por PCA. Se requieren 135 componentes para preservar el 90% de la varianza total.

La Figura 5 muestra ejemplos de imágenes originales junto con sus reconstrucciones obtenidas mediante PCA y Autoencoder. En ambos casos se observa que la reducción de dimensionalidad permite conservar la estructura general de las prendas, aunque parte de la información presente en las imágenes originales se pierde durante el proceso de compresión y reconstrucción.

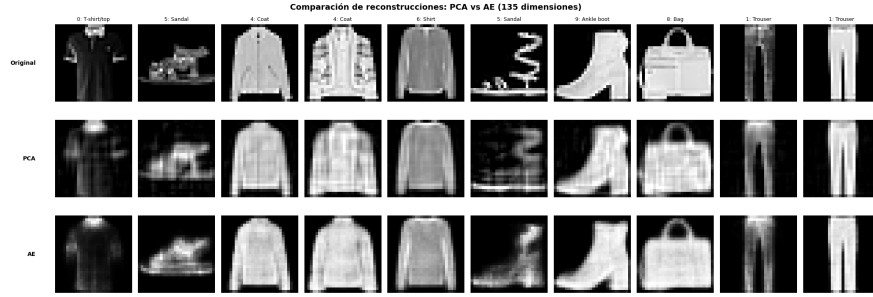


Figura 5: Comparación de imágenes originales (arriba) y reconstrucciones obtenidas mediante PCA (centro) y Autoencoder (abajo).

Las reconstrucciones obtenidas mediante PCA presentan una pérdida de detalle respecto de las imágenes originales, aunque mantienen correctamente la forma global de la mayoría de las prendas. De manera similar, el Autoencoder logra reconstruir las observaciones conservando las características visuales principales de cada objeto.

Para cuantificar la calidad de las reconstrucciones sobre el conjunto de evaluación, se calcularon el Error Cuadrático Medio (MSE), la Raíz del Error Cuadrático Medio (RMSE) y el Error Absoluto Medio (MAE). Los resultados obtenidos se presentan en la Tabla 1.

Cuadro 1: Comparación de errores de reconstrucción entre PCA y Autoencoder.

Métrica	PCA	Autoencoder
MSE	0.0081	0.0103
RMSE	0.0899	0.1016
MAE	0.0545	0.0520

Los resultados muestran que ambos métodos producen reconstrucciones de calidad similar. PCA obtuvo valores menores de MSE y RMSE, mientras que el autoencoder presentó un MAE ligeramente inferior. Dado

que los valores de intensidad de los píxeles se encuentran normalizados en el intervalo $[0, 1]$, las diferencias observadas entre ambos métodos resultan relativamente pequeñas.

En conjunto, estos resultados indican que tanto PCA como el autoencoder logran representar las imágenes utilizando únicamente 135 variables latentes y conservar gran parte de la información visual presente en los datos originales. Para esta configuración particular, PCA presentó una ligera ventaja según las métricas de reconstrucción.

3.3 Análisis de K

Con el objetivo de seleccionar una cantidad adecuada de clusters, se evaluó el desempeño de K-Means y GMM para valores de K comprendidos entre 5 y 15. Para ello se utilizaron dos criterios: la ganancia marginal obtenida al aumentar el número de clusters y el *Silhouette score*, que permite evaluar qué tan compactos y separados quedan los grupos obtenidos.

En el caso de K-Means, el análisis se realizó a partir de la inercia. Valores menores de inercia indican que las observaciones se encuentran más cerca de los centroides de sus respectivos clusters. Como las representaciones latentes obtenidas mediante PCA y Autoencoder no necesariamente se encuentran en la misma escala, además de la inercia absoluta se observó su versión normalizada, permitiendo comparar la mejora relativa obtenida al aumentar K .

La Figura 6a muestra que, tanto para PCA como para Autoencoder, la inercia disminuye al incrementar la cantidad de clusters. La mayor reducción se observa para los primeros valores de K , mientras que posteriormente las mejoras se vuelven más pequeñas. Esto indica que agregar nuevos clusters sigue mejorando el ajuste del modelo, aunque el beneficio obtenido es cada vez menor.

Para complementar este análisis se calcularon la ganancia marginal de inercia y el *Silhouette score*, mostrados en la Figura 7. La ganancia marginal alcanza sus valores más altos para los primeros incrementos de K y luego tiende a disminuir, lo que sugiere que cada nuevo cluster aporta menos información adicional que el anterior. Por otro lado, el mayor valor de *Silhouette score* sobre la representación obtenida mediante PCA se alcanza en $K = 8$. Esto indica que, en promedio, los puntos se encuentran relativamente bien agrupados dentro de sus clusters y suficientemente separados de los demás. Considerando ambas métricas, se seleccionó $K = 8$ para el algoritmo K-Means.

En GMM, el desempeño se evaluó mediante la log-verosimilitud del modelo. Esta métrica mide qué tan bien la mezcla de gaussianas ajustada logra representar los datos observados, por lo que valores más altos indican un mejor ajuste. Como puede observarse en Figura 6b, la log-verosimilitud aumenta al incrementar K tanto para PCA como para Autoencoder. Este comportamiento era esperable, ya que al agregar más componentes gaussianas el modelo dispone de una mayor flexibilidad para adaptarse a los datos.

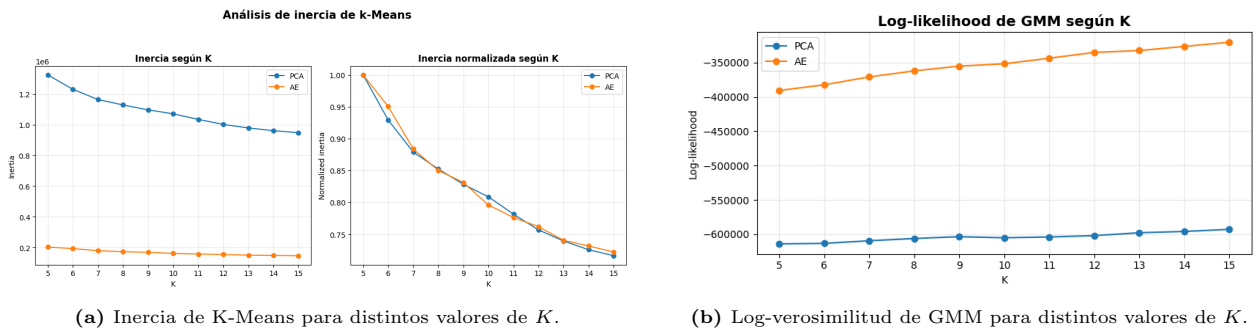


Figura 6: Evolución de las funciones objetivo utilizadas para evaluar distintos valores de K .

Sin embargo, una log-verosimilitud más alta no implica necesariamente una mejor estructura de clusters. Por este motivo también se analizó la ganancia marginal asociada a cada incremento de K (Figura 7). A diferencia de lo observado en K-Means, las mejoras resultan más irregulares y no muestran una tendencia tan clara. En consecuencia, la selección final se apoyó principalmente en el *Silhouette score*. Para la representación obtenida mediante PCA, los mejores valores se observan alrededor de $K = 8$ y $K = 11$. Dado que las

diferencias son pequeñas y con el objetivo de mantener consistencia con el análisis realizado para K-Means, se decidió utilizar $K = 8$ también para GMM.

Finalmente, es importante destacar que el valor seleccionado no coincide necesariamente con las diez clases originales presentes en Fashion-MNIST. Esto es esperable, ya que los algoritmos de clustering no utilizan las etiquetas reales durante el entrenamiento, sino que agrupan observaciones según las similitudes presentes en el espacio de representación. Como consecuencia, categorías visualmente parecidas pueden terminar dentro de un mismo cluster, mientras que clases con una alta variabilidad interna pueden distribuirse entre varios grupos distintos.

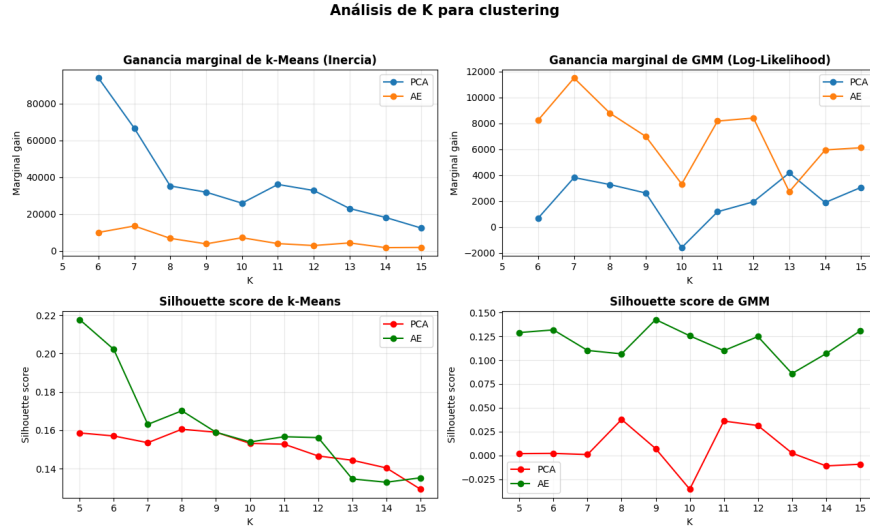


Figura 7: Curvas de Silhouette score y ganancia marginal para K-Means (inercia) y GMM (log-likelihood) tanto para la representación obtenida mediante de PCA y autoencoder.

3.4 Calidad de los clusters

Una vez seleccionado $K = 8$, se analizó la calidad de los clusters obtenidos para el grupo de entrenamiento sampleado correspondiente a la reconstrucción de PCA. Se consideraron tres aspectos: el tamaño de cada grupo, la composición de clases reales dentro de cada cluster y su visualización en dos dimensiones mediante t-SNE.

En primer lugar, se observa que K-Means genera clusters relativamente más balanceados que GMM, como se observa en la Figura 8. En K-Means, la mayoría de los grupos contienen una buena proporción de las observaciones, aunque existen algunos clusters más pequeños asociados a categorías muy específicas. Por el contrario, GMM presenta una distribución más dispareja, con componentes que concentran una gran cantidad de muestras y otras que contienen una fracción reducida de los datos.

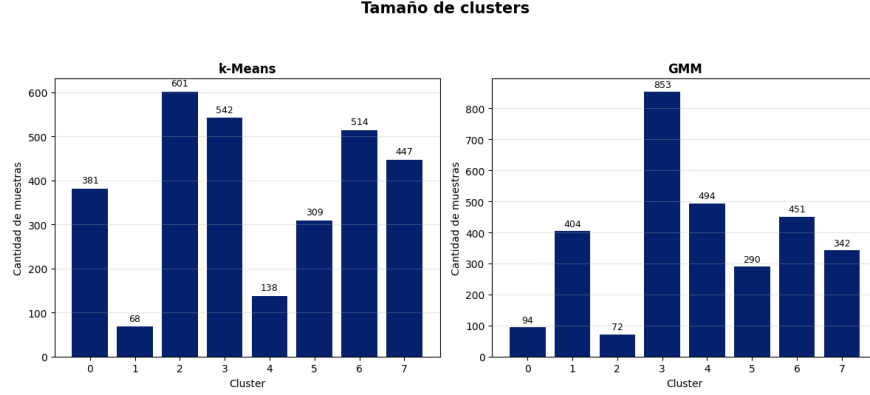


Figura 8: Tamaño de los clusters obtenidos con K-Means y GMM para $K = 8$. Se observa una distribución más balanceada en K-Means que en GMM.

Para complementar el análisis del tamaño de clusters, se calculó la pureza de cada cluster. Esta métrica indica la proporción de observaciones pertenecientes a la clase más frecuente dentro de un grupo. Valores cercanos a 1 corresponden a clusters más homogéneos, mientras que valores bajos indican una mayor mezcla de categorías. La Tabla 2 resume la clase dominante y la pureza asociada a cada cluster para K-Means y GMM.

En K-Means, por ejemplo, el cluster 4 agrupa principalmente bolsos, alcanzando una pureza de 0.935, mientras que el cluster 1 concentra mayoritariamente botas con una pureza de 0.824. También aparecen clusters relativamente homogéneos asociados a zapatillas y pantalones. En contraste, otros grupos contienen una mezcla considerable de prendas superiores como remeras, camisas, pulóveres y abrigos.

GMM muestra un comportamiento similar. Algunas componentes presentan niveles de pureza elevados, como el cluster 0, dominado por bolsos con una pureza de 0.947, o el cluster 7, compuesto principalmente por pantalones. Sin embargo, las componentes de mayor tamaño mezclan varias categorías simultáneamente. Esto ocurre especialmente con las prendas superiores, donde remeras, camisas, pulóveres y abrigos aparecen distribuidos en los mismos clusters.

Cuadro 2: Comparación de la pureza de los clusters obtenidos mediante K-Means y GMM.

K-Means			GMM		
Cluster	Clase	Pureza	Cluster	Clase	Pureza
0	T-shirt/top	0.483	0	Bag	0.947
1	Ankle boot	0.824	1	Sneaker	0.604
2	Shirt	0.208	2	Bag	0.486
3	Coat	0.349	3	Ankle boot	0.192
4	Bag	0.935	4	T-shirt/top	0.304
5	Ankle boot	0.712	5	Coat	0.434
6	Sneaker	0.543	6	Ankle boot	0.231
7	Trouser	0.573	7	Trouser	0.681

Estos resultados son consistentes con el análisis exploratorio realizado anteriormente. Las imágenes promedio mostraban que categorías como bolsos, pantalones, zapatillas y botas poseen siluetas claramente diferenciadas, mientras que las distintas prendas superiores comparten formas generales similares. En consecuencia, resulta esperable que los algoritmos logren separar con mayor facilidad las primeras y presenten mayores dificultades para distinguir las segundas. Para más resultados y gráficos obtenidos visitar la sección 3.d del Notebook.

La Figura 9 muestra la proyección t-SNE de las representaciones latentes obtenidas mediante PCA junto con las asignaciones realizadas por K-Means y GMM. En ambos casos puede observarse cierta estructura en los datos, con regiones donde predominan determinados clusters. Sin embargo, también existen zonas de superposición importantes entre grupos, especialmente en las regiones centrales del espacio proyectado.

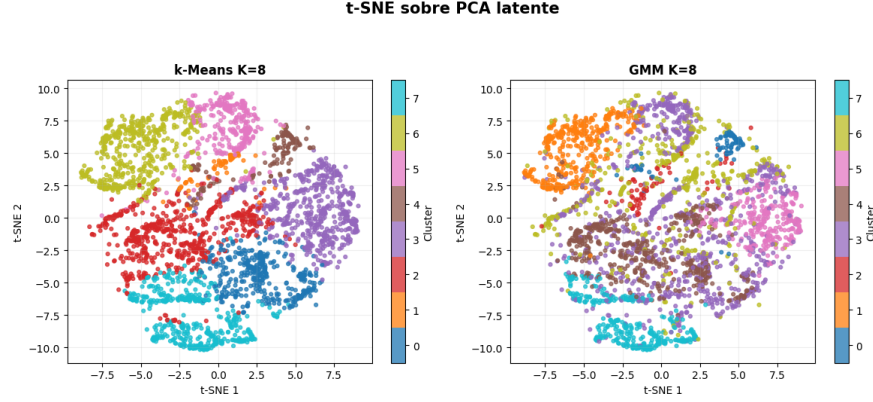


Figura 9: Proyección bidimensional mediante t-SNE de las representaciones latentes obtenidas con PCA. Los colores indican la asignación de clusters realizada por K-Means y GMM para $K = 8$. Se observan regiones parcialmente separadas junto con zonas de superposición entre grupos.

En términos generales, ambos algoritmos logran capturar parte de la estructura presente en los datos, identificando categorías visualmente distintivas y agrupando observaciones con características similares. No obstante, los clusters obtenidos no reproducen de forma exacta las clases reales del dataset. Esto es esperable debido a la gran similitud entre algunas categorías, provocando que caigan muy cerca entre sí y los algoritmos las interpreten como un solo cluster. Lo importante es ver que las prendas similares son puestas en un mismo cluster, y no está agrupando muestras muy distintas.

4. Conclusión

En este trabajo se aplicaron técnicas de reducción de dimensionalidad y clustering sobre el dataset Fashion-MNIST. Se compararon dos métodos de reducción de dimensionalidad: PCA y un Autoencoder determinístico implementado en PyTorch. Utilizando un criterio de preservación del 90% de la varianza, PCA permitió reducir las imágenes de 784 a 135 dimensiones.

Las métricas de reconstrucción mostraron resultados similares para ambos enfoques, observándose una ligera ventaja de PCA en términos de MSE y RMSE. Esto indica que es posible lograr una compresión significativa de los datos conservando gran parte de la información visual relevante.

Sobre las representaciones reducidas se aplicaron K-Means y GMM. El análisis de inercia, log-verosimilitud, ganancias marginales y *Silhouette score* llevó a seleccionar $K = 8$ para ambos métodos, sugiriendo que la estructura natural de los datos no coincide necesariamente con las diez categorías originales del dataset.

Los clusters obtenidos lograron identificar con relativa claridad categorías visualmente distintivas como bolsos, pantalones y distintos tipos de calzado. En contraste, las prendas superiores presentaron una mayor superposición, generando agrupamientos más heterogéneos. En conjunto, los resultados muestran que las técnicas de aprendizaje no supervisado permiten descubrir patrones relevantes y estructuras con significado visual, aunque sin reproducir exactamente las clases originales del conjunto de datos.